

Name: Dau Vu Tuan Khoi

ID: 105709548

Technical Report Task C.2: Optimizing Data Processing and Visualization

1. Overview of Task C.2

Task C.2 focuses on restructuring and upgrading the data processing process (Data Pipeline) from stock_prediction source code. py v0.1. The goal is to address data feed performance limitations, improve input characteristic quality, and overcome gaps in the standardization process.

Key improvement items include:

Efficient data loading and storage: Build a local caching mechanism.

Mid-point testing: Evaluating the effectiveness of the average price compared to the closing price alone.

Adjust the time parameter: Optimize the look-back window via PREDICTION_DAYS .

Fix the normalization error: Thoroughly handle the phenomenon of out-of-distribution year-values.

Visualization Upgrade: Convert to a dedicated candlestick chart to analyze market movements.

Note: According to preliminary assessments, the v0.1 model currently has a next-day forecast error ranging from 10% to 13%. The improvements in Task C.2 are the premise to minimize this error.

2. Improved Data Loading and Storage Process

Based on the requirements in the Load Data block, we converted the data loading method from "direct load" to a conditional mechanism to optimize resources and avoid API (rate-limiting).

Technical Directive:

Implement the test function: Use the os library (os. path. exists function) to check the existence of the CSV file corresponding to the ticker code (COMPANY) in the working directory.

Data Processing Flow:

Track 1 (Data Exists): Use pandas. read_csv() to load data from disk directly into memory, skipping the download step from the Internet.

Branch 2 (Non-existent data): Use yfinance (yf . down load) to retrieve data from Yahoo Finance servers, then immediately execute df. to_csv() for local storage.

Benefits: Reduced latency when running multiple tests and protecting data integrity in the event of network connectivity loss.

3. Price Mid-point Testing

Instead of relying solely on the closing price - which only reflects the final state of the trading session, we implemented the test with the mid-point price characteristic.

Calculation logic: Create a new column in the DataFrame:

$$\text{Mid-point} = (\text{Open} + \text{Close}) / 2.$$

Why use Mid-point: This feature provides a more holistic view of the average trading value of the day, reducing noise from extreme fluctuations at the time of closing the session.

Implementation: This calculation must be performed before the data is included in the MinMaxScaler suite. PRICE_VALUE variables will be assigned with this new column name as input to the entire training and testing process.

4. Parameter Adjustment PREDICTION_DAYS

The PREDICTION_DAYS parameter is essentially a sliding window strategy that determines the amount of past data that the LSTM model needs to forecast the future.

Impact on data structure:

The number of data samples (p) is calculated according to the formula:

$$p = \text{len}(\text{scaled_data}) - \text{PREDICTION_DAYS}.$$

The input banana length (q) is the value of PREDICTION_DAYS.

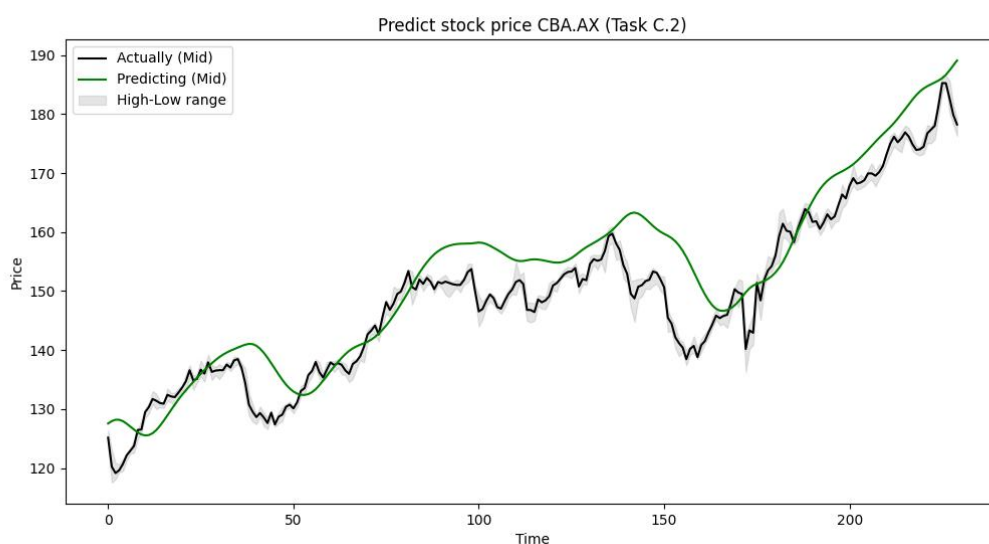
5. Upgrade Visualization with Candlestick Charts

The line chart alone in v0.1 does not reflect market sentiment and volatility during the session.

Implementation requirements: Use OHLC (Open, High, Low, Close) data obtained from the yf command. download.

Value Analysis: Candlestick charts allow the engineer to observe the difference between the high/low price compared to the model's forecast.

Integration: The new chart should display candlesticks (strike price) and lines (forecast price) simultaneously on the same coordinate axis. This helps identify whether the pattern is struggling at sideways market periods or in high volatility sessions.



6. Technical documents

During the implementation of C.2, I spent time researching techniques and professional forums such as StackOverflow, Machine Learning Mastery, GitHub,...

Specifically, as follows:

- 1 mechanism in row reshape:

```
scaled_data = scaler.fit_transform(data[PRICE_VALUE].values.reshape(-1, 1))
```

The sklearn library requires the input data for MinMaxScaler to be a 2-dimensional array of form (n_samples, n_features). However, the raw data from pandas is usually 1-dimensional arrays. The parameter -1 in numpy is a wildcard. It requires the library to automatically calculate the size of that dimension based on the total number of elements of the array and the size of the remaining dimensions defined. This makes the code more flexible as the dataset size changes.

- Return_sequences=True when using LSTM

```
model = Sequential([
    LSTM(units=50, return_sequences=True,
input_shape=(x_train.shape[1], 1)),
    Dropout(0.2),
    LSTM(units=50, return_sequences=True),
    Dropout(0.2),
    LSTM(units=50),
    Dropout(0.2),
    Dense(units=1)
])
```

In a stacked LSTM architecture, the intermediate LSTM layers need to transmit the entire data chain to the next layer instead of just transmitting the final result. If you set return_sequences = True (default), the LSTM class will return only one 2-dimensional vector. This will give an error because the input must be a 3D sequence consisting of samples, timestep, and features.

- Convert data to 3D arrays for LSTM:

```
x_test = np.reshape(x_test, (x_test.shape[0], x_test.shape[1], 1))
```

LSTM requires the input to be 1 3-way array. The first dimension is the number of samples. The 2nd dimension is the number of time steps (time_steps) in the source code of this task is PREDICTION_DAYS. The 3rd dimension is the number of features. Without this picture, the model will not be able to carry out the training process.